

Práctica 3 - Planificación de caminos.

Alejandro Otero Vázquez y Mario Bravo López

Fecha de realización: 13 de enero de 2025

1. Objetivos.

El punto de partida desde el que se comienza a construir esta práctica, tras la anterior, es el siguiente. Hasta ahora se ha construido un robot, de un tamaño aproximado de 20 cm x 15 cm capaz de moverse en línea recta y girar. Gracias a unos sensores infrarrojo el robot también es capaz de detectar objetos imprevistos que obstaculicen su camino, pero no es capaz de esquivarlos.

El objetivo de esta práctica será el de conseguir que el control del robot se realice a través de funciones que tengan como parámetros de entrada las posiciones inicial y final del robot y no las distancias rectilíneas y giros que debe hacer el robot para moverse. Es decir, en este documento se detallarán los procedimientos que se han llevado a cabo para construir algoritmos que descodifiquen posiciones en el espacio en instrucciones de movimiento para el robot.

El objetivo final será que el robot sea capaz de esquivar obstáculos imprevistos que aparezcan durante su trayectoria y sea capaz aún así de alcanzar su posición final.

2. Consideraciones iniciales.

Hasta ahora, el movimiento del robot estaba gobernado por las funciones:

- `void motorMoveRelative(int security, float mm, void *args_ptr);`
- `void motorStop();`
- `void motorRotateAngle(float angle, void *args_ptr);`

Donde `void *args_ptr` es una estructura que contiene todas las variables compartidas entre los hilos. El significado del resto de variables puede deducirse de su propio nombre.

El cambio que se introduce en esta práctica es que las funciones de movimiento ahora tendrán como parámetro de entrada también un puntero a una estructura que contendrá las coordenadas actuales del robot. Las propias funciones de movimiento modificarán las coordenadas del robot a medida que se ejecuten accediendo a la dirección de memoria de la estructura. Esta funcionalidad permite conocer la posición del robot en un momento dado en que se hayan ejecutado una serie de instrucciones para así poder recalcular la ruta que le lleve a la posición de destino. La estructura Posición que contiene las coordenadas del robot es la siguiente:

```
typedef struct{
float x;
float y;
float theta;
} Position;
```

Las funciones de movimiento por tanto ahora tendrán la siguiente estructura:

- `void motorMoveRelative(int security, float mm, void *args_ptr, Position *actualPosition);`

- `void motorStop();`
- `void motorRotateAngle(float angle, void *args_ptr, Position *actualPosition);`

Cabe destacar que aunque la función `motorMoveRelative` contemple una distancia de seguridad para detectar objetos cercanos, esta variable permanecerá a 0 durante los Experimentos 1-3, donde el robot no tendrá en cuenta obstáculos.

2.1. `void motorMoveRelative();`

Las nuevas funcionalidades que incluye la función son una rutina de esquivar de obstáculos imprevistos y la actualización de las coordenadas del robot cuando se termina de ejecutar la función.

2.1.1. Esquiva de obstáculos.

1. La función comienza verificando si la distancia al obstáculo detectada por el sensor izquierdo (`distLeft`) o el sensor derecho (`distRight`) es menor o igual que un valor de seguridad predefinido (`security`).
2. Si se detecta un obstáculo cercano en cualquiera de los lados, se emite un mensaje de advertencia y se activa una acción de movimiento de precaución. Esto implica reducir la velocidad de los motores (`pwmWrite`) y esperar un corto período de tiempo (`delay`) para permitir la reevaluación de la situación.
3. Después de la acción de precaución inicial, se inicia un bucle `do-while` que se ejecutará mientras se cumplan ciertas condiciones. Estas condiciones incluyen:
 - a) La cantidad de cuentas del codificador (`counterLeft` y `counterRight`) no hayan alcanzado los valores de cuentas necesarios para haber recorrido la distancia objetivo.
 - b) La distancia a los obstáculos (`distLeft` y `distRight`) sigue siendo mayor o igual a 15 centímetros.
4. Si después de moverse, todavía se detecta que el obstáculo está demasiado cerca (menos de 15 centímetros), se activa una rutina de esquivar (función auxiliar `motorDodgeRoutine()`) para maniobrar alrededor del obstáculo. Esto implica la modificación de las coordenadas del vehículo (función auxiliar `modifyCoordinates()`).

2.1.2. Actualización de coordenadas

La función `void modifyCoordinates(float x, float y, float theta, Position *position)` accede a la dirección de memoria de una estructura posición y actualiza los valores que almacena a los valores que tiene como entrada la función. Cuando se ejecuta esta función dentro de `motorMoveRelative()` los valores de `x` y `y` son los siguientes (`theta` permanece invariante):

- `float x = actualPosition->x+((float)(counterLeft)+(float)(counterRight))*M_PI diametro/40*sin(actualPosition->theta*M_PI/180);`
- `float y = actualPosition->y+((float)(counterLeft)+(float)(counterRight))*M_PI diametro/40*cos(actualPosition->theta*M_PI/180);`

En resumen, esta función maneja la detección de obstáculos cercanos y la toma de medidas para evitar colisiones mediante el ajuste de la velocidad y la ejecución de maniobras evasivas cuando sea necesario.

2.2. `void motorRotateAngle();`

La nueva funcionalidad que incluye la función es la actualización de las coordenadas del robot cuando se termina de ejecutar la función. El proceso es similar al de `motorMoveRelative`, cuando termina de ejecutarse el movimiento, se ejecuta la función auxiliar `modifyCoordinates()` con valor de entrada de `theta`:

- `float theta = actualPosition->theta+angle/abs(angle)*((float)(counterLeft)/2+(float)(counterRight)/2)*360*diametro/cuentas/eje;`

3. Funciones auxiliares.

3.1. Position *definePosition();

La función `*definePosition()` reserva memoria dinámica para reservar la suficiente cantidad de memoria como para almacenar la información de la estructura `Position`. Seguidamente inicializa las coordenadas del punto a definir en el origen de coordenadas. Finalmente retorna la estructura con sus valores de coordenadas inicializados.

Cada vez que se quiera definir un nuevo punto, deberá hacerse igualando la variable `Position` creada a esta función. De este modo se reservará la memoria necesaria para almacenar los datos en memoria y se inicializarán las coordenadas. Todas las variables `Position` que se definan, serán punteros. De este modo, cuando las funciones reciban como argumento un puntero a una variable `Position` podrán acceder y modificar directamente los datos almacenados en la dirección de memoria de la estructura `Position`.

Esta forma de trabajar permite a las funciones actualizar siempre la posición del robot accediendo a la dirección de memoria que almacena sus coordenadas. Además, esto hace que cuando una función sucesiva acceda a las coordenadas del robot verá las coordenadas actualizadas por la función que la precede. Haciendo así que la posición del robot sea conocida por todas las funciones, permitiendo así que se recalcule la trayectoria del motor en tiempo real.

```
Position *definePosition(){
Position *r = malloc(sizeof(Position));
if(r==NULL){
exit(EXIT_FAILURE);
}
r->x = 0.0;
r->y = 0.0;
r->theta = 0.0;
return r;
}
```

3.2. void modifyCoordinates(float x, float y, float theta, Position *position);

La función `modifyCoordinates()` accede a la dirección de memoria de una estructura `Posición`, que almacena unas coordenadas y las modifica. La función establecerá las coordenadas del punto como `float x, float y, float theta`.

3.3. void motorDodgeRoutine(Position *actualPosition, void *args_ptr);

La función `motorDodgeRoutine` ejecuta una rutina de esquiva de objetos predeterminada. En caso de que se haya detectado un objeto se llamará a esta función. Esta rutina de esquiva consiste en la siguiente secuencia de movimientos:

```
motorRotateAngle(-90, args_ptr, actualPosition);
motorMoveRelative(0, 400, args_ptr, actualPosition);
motorRotateAngle(90, args_ptr, actualPosition);
motorMoveRelative(0, 500, args_ptr, actualPosition);
```

Hay que tener en cuenta que dentro de sí, las funciones `motorRotateAngle()` y `motorMoveRelative()` actualizarán las coordenadas del robot teniendo en cuenta el número de cuentas que hayan medido los encoders.

3.4. void generateInstructions(Position *actualPosition, Position *nextPosition, void *args_ptr, int security, int *ptr);

La función `generateInstructions` recibe las coordenadas de dos posiciones denominadas `*actualPosition` y `*nextPosition` además de un puntero a los argumentos compartidos por los hilos `*args_ptr`. A este pun-

tero accederán las funciones de movimiento para obtener información acerca de los sensores. Esta función calcula el giro que debe realizar el motor para apuntar a la dirección donde esté el punto `nextPosition`. Seguidamente recorrerá en línea recta la distancia entre los dos puntos a partir del teorema de Pitágoras, siendo los catetos las diferencias en x e y entre los puntos. Finalmente, si el argumento `*ptr` no es nulo, el robot girará hasta alcanzar la orientación definida por el punto `nextPosition`.

4. Software de planificación de caminos.

Una vez definidas y aclaradas las funciones auxiliares que se han implementado para la mejora del software de cara a realizar la tercera práctica. Vamos a analizar en el contexto de cada experimento como se usan estas funciones para poder cumplir los objetivos exigidos por estos apartados. Hay que aclarar que el hilo que se modificará será el asociado con el movimiento, `void* movement(void* args_ptr)`, en el archivo entregado `Robot_P3.c`, se deben descomentar las líneas asociadas a cada experimento para su correcto uso.

4.1. Experimento 1. Cumplimiento de órdenes de movimiento:

Para este apartado debemos conseguir que el robot sea capaz de seguir una ordenes de movimiento definidas en un array del tipo: `char commands[] = "D800, R-45, D400"`; En el cual se hace referencia al tipo de movimiento que se desea realizar usando, R para rotación y D para desplazamiento, seguido de la cantidad que se desea rotar o avanzar.

El primer paso que se realiza es la separación de los elementos de la cadena de caracteres `comands[]`, usando un delimitador y la función: `token=strtok(commands,delimiter)`, estas cadenas ya separadas se analizan usando `sscanf(token, "%c%d", &letra, &valor)`, consiguiendo almacenar en `letra` el tipo de movimiento a realizar y en `valor` su cantidad. Usando una estructura de control de tipo `switch(){case:...}` se ejecutan las funciones: `motorMoveRelative()` y `motorRotateAngle()`, basados en la letra que se haya detectado en las cadenas de caracteres.

4.2. Experimento 2. Planificación de movimiento sin mapa

Para este experimento se pide el uso de un sistema de coordenadas cartesianas, para realizar desplazamientos entre un punto inicial con coordenadas y orientación (x_0, y_0, θ_0) y un punto final (x_1, y_1, θ_1) , con el objetivo de que se alcance este punto y su correspondiente orientación.

En este experimento necesitaremos definir dos punteros a estructuras de tipo `Position`, usando la función `definePosition()`. La primera de ellas será la posición desde la que se supone que se está partiendo **actualPosition** y la segunda será el punto al que se desea llegar **nextPosition**. Usando la función auxiliar `modifyCoordinates()`, se pasaran como variables de entrada las coordenadas que se desean asociar a la posición inicial que en general siempre se inicializará al origen de coordenadas: `modifyCoordinates(0,0,0, actualPosition)`; y las coordenadas y orientación que deseamos alcanzar. Una vez definidas correctamente las estructuras se usa la función auxiliar `generateInstructions(actualPosition, nextPosition, args_ptr, 0, &ptr_value)`, con el objetivo de calcular los movimientos de rotación y avance necesarios a realizar para llevar a cabo el desplazamiento entre los puntos definidos.

4.3. Experimento 3. Seguimiento de trayectorias complejas

Para este apartado se pide el seguimiento de una trayectoria del tipo `float camino[]={x1, y1, x2, y2, x3, y3}`, por parte del robot. Variando de manera sutil el código del apartado anterior podemos conseguir la implementación pedida. El primer paso será evidentemente, definir dos punteros a estructuras `Position`, la posición actual y la posición a la que se desea llegar. Se inicializa la primera posición, al origen de coordenadas: `modifyCoordinates(0,0,0, actualPosition)`. Y se procede a realizar un bucle en el cual se van modificando las coordenadas de la estructura `nextPosition`, con los valores del array `camino`, para posteriormente usando la función `generateInstructions(actualPosition, nextPosition, args_ptr, 0,`

`&ptr_value`) calcular y ejecutar los movimientos necesarios para desplazarse entre un punto y otro. Hay que destacar que en esta última función auxiliar, al finalizar el desplazamiento se actualiza la variable `actualPosition` a la posición a la que se ha llegado, consiguiendo así que el bucle funcione de manera correcta.

4.4. Experimento 4. Esquiva de obstáculos imprevistos

En este último experimento se debe realizar un código muy similar al implementado en el segundo experimento, pero con la novedad de que se debe implementar una rutina de esquiva de obstáculos. Por ende se deberá usar de manera activa los sensores de distancia Sharp. Respecto al código, la única diferencia es el uso de una distancia de seguridad en el cuarto argumento de la función `generateInstructions(actualPosition, nextPosition, args_ptr, 20, &ptr_value)`. En este caso la distancia de seguridad se ha definido en 20 *cm*, pero puede modificarse a gusto por el usuario. Este valor para la distancia será la que tendrá en cuenta la función `motorMoveRelative()`, para detener el avance y ejecutar la rutina implementada en la función `motorDodgeRoutine()`.

5. Código Usado

Por petición del profesor de la asignatura se incluye el código usado para el desarrollo de la práctica.

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/time.h>

#include <wiringPi.h>
#include <wiringPiI2C.h>
#include <string.h>

#include <wiringPiSPI.h>
#include <mcp3004.h>
#include <math.h>

#define spi_channel 0
#define pbDstLft 101
#define pbDstRth 100
#define pbCntLft 103
#define pbCntRth 102

#define velocidad 5e4// 1 MHz

// #define MAX_ROWS 70
#define MAX 10
#define LSM6DS33 0x6a

#define Pin_D 23 // BCM 13, este pin es el que ira en sentido antihorario para moverse hacia adelante
#define Pin_I 1 // BCM 18, este ira en sentido horario, pues es la salida para el motor derecho

float cuentas = 20;
float diametro = 71;
float eje = 198;
int dimension;
```

```

typedef struct {
    float distLeft;
    float distRight;
    int counterLeft;
    int counterRight;
    pthread_mutex_t mutexLeft;
    pthread_mutex_t mutexRight;
    pthread_mutex_t mutexEncoderLeft;
    pthread_mutex_t mutexEncoderRight;
    float *x;
    float *voltageLft;
    float *voltageRht;
    pthread_barrier_t barrera;
    int salir;
} ThreadArgs;

typedef struct{
    float x;
    float y;
    float theta;
} Position;

void motorDodgeRoutine(Position *actualPosition, void *args_ptr);
void modifyCoordinates(float x, float y, float theta, Position *position);
void motorMoveRelative(int security, float mm, void *args_ptr, Position *actualPosition);

// Función para inicializar la estructura compartida
ThreadArgs* initSharedData() {
    ThreadArgs *data = malloc(sizeof(ThreadArgs));
    if (data == NULL) {
        fprintf(stderr, "Error de asignación de memoria\n");
        exit(EXIT_FAILURE);
    }
    data->distLeft = 0.0;
    data->distRight = 0.0;
    data->counterLeft = 0;
    data->counterRight = 0;
    data->salir = 0;
    pthread_mutex_init(&data->mutexLeft, NULL);
    pthread_mutex_init(&data->mutexRight, NULL);
    pthread_mutex_init(&data->mutexEncoderLeft, NULL);
    pthread_mutex_init(&data->mutexEncoderRight, NULL);
    pthread_barrier_init(&data->barrera, NULL, 3);
    return data;
}

void destroySharedData(ThreadArgs *data) {
    pthread_mutex_destroy(&data->mutexLeft);
    pthread_mutex_destroy(&data->mutexRight);
    pthread_mutex_destroy(&data->mutexEncoderLeft);
    pthread_mutex_destroy(&data->mutexEncoderRight);
    pthread_barrier_destroy(&data->barrera);
}

```

```

    free(data->x);
    free(data->voltageLft);
    free(data->voltageRht);
    free(data);
}

float getDistance(float measure, float *x, float *voltage){
    float dist;
    int n = 0;
    while(n<dimension){
        if(measure< *voltage){
            x++;
            voltage++;
        }
        n++;
    }
    float interval = *voltage-*(voltage-1);
    float frac = (measure-*(voltage-1))/interval;
    //printf("%f\n",interval);
    dist = *(x-1) + frac*(x-*(x-1));
    return dist;
}

/*void chargeValue(float *x, float *voltage){
    FILE *file;
    file = fopen("calibracion.csv","r");

    if (file == NULL) {
        printf("No se pudo abrir el archivo\n");
        exit(1);
    }

    // Leer el archivo CSV
    while (fila < MAX_ROWS) {
        fscanf(file,"%f;%f",x,voltage);
        printf("%lf\t%lf\n",*x, *voltage);
        x++;
        voltage++;
    }
    fclose(file);
}*/

void calDistance(ThreadArgs *data){
    int precision;
    printf("Calibrando el vehículo. Por favor espere...\n");
    printf("Qué precisión en cm quiere que tenga la calibración: \n");
    scanf("%d",&precision);
    dimension = (int)(30/precision+1);
    printf("Dimension: %d\n",dimension);
    data->voltageLft = malloc(dimension* sizeof(float));
    if (data->voltageLft == NULL) {
        // Manejo de error si malloc falla
        perror("Error al asignar memoria para el array");
    }
}

```

```

exit(EXIT_FAILURE);
}
data->voltageRht = malloc(dimension * sizeof(float));
if (data->voltageRht == NULL) {
// Manejo de error si malloc falla
perror("Error al asignar memoria para el array");
exit(EXIT_FAILURE);
}
data->x = malloc(dimension * sizeof(float));
if (data->x == NULL) {
// Manejo de error si malloc falla
perror("Error al asignar memoria para el array");
exit(EXIT_FAILURE);
}

pthread_mutex_lock(&data->mutexEncoderLeft);
data->counterLeft=0;
pthread_mutex_unlock(&data->mutexEncoderLeft);

pthread_mutex_lock(&data->mutexEncoderRight);
data->counterRight=0;
pthread_mutex_unlock(&data->mutexEncoderRight);
int n, counterLeft, counterRight, counter,j;
float t1;
float measure[2] = {0.0,0.0};
j=0;
for(int i=precision;i<=30;i+=precision){
n=0;

counter = (int)cuentas*i*10/(M_PI*diametro);
//printf("Counter: %d\n",counter);
pwmWrite(Pin_D,320);
pwmWrite(Pin_I,280);

do
{
pthread_mutex_lock(&data->mutexEncoderLeft);
counterLeft = data->counterLeft;
pthread_mutex_unlock(&data->mutexEncoderLeft);

pthread_mutex_lock(&data->mutexEncoderRight);
counterRight = data->counterRight;
pthread_mutex_unlock(&data->mutexEncoderRight);
} while (counterLeft <= counter || counterRight <= counter);
if(counterLeft<counterRight){
pwmWrite(Pin_D,300);
pwmWrite(Pin_I,280);
}
do
{
pthread_mutex_lock(&data->mutexEncoderLeft);
counterLeft = data->counterLeft;
pthread_mutex_unlock(&data->mutexEncoderLeft);

```



```

pthread_mutex_lock(&data->mutexEncoderRight);
counterRight = data->counterRight;
pthread_mutex_unlock(&data->mutexEncoderRight);
} while (counterLeft < counterRight);
if(counterLeft>counterRight){
pwmWrite(Pin_D,320);
pwmWrite(Pin_I,300);
}
do
{
pthread_mutex_lock(&data->mutexEncoderLeft);
counterLeft = data->counterLeft;
pthread_mutex_unlock(&data->mutexEncoderLeft);

pthread_mutex_lock(&data->mutexEncoderRight);
counterRight = data->counterRight;
pthread_mutex_unlock(&data->mutexEncoderRight);
} while (counterLeft > counterRight);

pwmWrite(Pin_I,300);
pwmWrite(Pin_D,300);
printf("Midiendo a %d cm...\n",i);

t1 = millis();
while(millis()-t1<5e3){
measure[0] += analogRead(pbDstLft)*3.3/1023;
measure[1] += analogRead(pbDstRth)*3.3/1023;
delay(100);
n++;
}

data->voltageRht[j] = measure[1]/n;
data->voltageLft[j] = measure[0]/n;
measure[0] = 0.0; measure[1] = 0.0;
printf("VL: %lf\tVR: %lf\tx: %d\n",data->voltageRht[j],data->voltageLft[j],i);
data->x[j] = i;
j++;
if(j>dimension)printf("Tamaño de array insuficiente\n");
}
data->voltageRht[j] = 0.0;
data->voltageLft[j] = 0.0;
data->x[j] = 40;
printf("Retire el objeto de calibración\n");
delay(6000);

pthread_mutex_lock(&data->mutexEncoderLeft);
data->counterLeft=0;
pthread_mutex_unlock(&data->mutexEncoderLeft);

pthread_mutex_lock(&data->mutexEncoderRight);
data->counterRight=0;
pthread_mutex_unlock(&data->mutexEncoderRight);

```

```
}
```

```
void *distanceMeasureRigth(void *args_ptr){
```

```
ThreadArgs *args = (ThreadArgs*)args_ptr;
```

```
pthread_barrier_wait(&args->barrera);
```

```
float *x = args->x;
```

```
float *voltage = args->voltageRht;
```

```
//SharedData *sharedData = (SharedData *)sharedData;
```

```
float t_muestreo=1e3/(3); //Pasamos el tiempo de muestreo a microsegundos para compararlo con lo
```

```
float measure[2];
```

```
float alpha = 0.9;
```

```
float t1,t2;
```

```
float distRight;
```

```
while(1) {
```

```
t1 = millis();
```

```
measure[0] = measure[1];
```

```
measure[1] = analogRead(pbDstRth)*3.3/1023; //Paso a voltios
```

```
measure[1] = alpha*measure[1]+(1-alpha)*measure[0];
```

```
distRight = getDistance(measure[1],x,voltage);
```

```
pthread_mutex_lock(&args->mutexRight);
```

```
args->distRight = distRight;
```

```
pthread_mutex_unlock(&args->mutexRight);
```

```
t2 = millis();
```

```
if(t_muestreo>(t2-t1))
```

```
delay(t_muestreo-(t2-t1)); //EL tiempo entre medida y medida menos lo que tardamos en leer el pi
```

```
if(args->salir) pthread_exit(NULL);
```

```
}
```

```
return (void *) NULL;
```

```
}
```

```
void *distanceMeasureLeft(void *args_ptr){
```

```
ThreadArgs *args = (ThreadArgs*)args_ptr;
```

```
pthread_barrier_wait(&args->barrera);
```

```
float *x = args->x;
```

```
float *voltage = args->voltageLft;
```

```
//SharedData *sharedData = (SharedData *)sharedData;
```

```
float t_muestreo=1e3/(3); //Pasamos el tiempo de muestreo a microsegundos para compararlo con lo
```

```
float measure[2];
```

```
float alpha = 0.9;
```

```
float t1,t2;
```

```

float distLeft;

while(1) {
t1 = millis();

measure[0] = measure[1];
measure[1] = analogRead(pbDstLft)*3.3/1023; //Paso a voltios
measure[1] = alpha*measure[1]+(1-alpha)*measure[0];

distLeft = getDistance(measure[1],x,voltage);
pthread_mutex_lock(&args->mutexLeft);
args->distLeft = distLeft;
pthread_mutex_unlock(&args->mutexLeft);

t2 = millis();
if(t_muestreo>(t2-t1))
delay(t_muestreo-(t2-t1)); //EL tiempo entre medida y medida menos lo que tardamos en leer el pi
if(args->salir) pthread_exit(NULL);
}

return (void *) NULL;
}

void motorMoveRelative(int security, float mm, void *args_ptr, Position *actualPosition){

printf("Moviendo el vehículo en linea recta %f mm. El vehiculo se detendrá si se detecta un
obstáculo a menos de %d cm.\n",mm, security);
ThreadArgs *data = (ThreadArgs*)args_ptr;

float distLeft, distRight;
int counterLeft, counterRight;
int counter = (int)cuentas*mm/(M_PI*diametro);

pthread_mutex_lock(&data->mutexEncoderLeft);
data->counterLeft=0;
pthread_mutex_unlock(&data->mutexEncoderLeft);

pthread_mutex_lock(&data->mutexEncoderRight);
data->counterRight=0;
pthread_mutex_unlock(&data->mutexEncoderRight);

do
{
pthread_mutex_lock(&data->mutexLeft);
distLeft = data->distLeft;
pthread_mutex_unlock(&data->mutexLeft);

pthread_mutex_lock(&data->mutexRight);
distRight = data->distRight;
pthread_mutex_unlock(&data->mutexRight);

pthread_mutex_lock(&data->mutexEncoderLeft);

```

```

counterLeft = data->counterLeft;
pthread_mutex_unlock(&data->mutexEncoderLeft);

pthread_mutex_lock(&data->mutexEncoderRight);
counterRight = data->counterRight;
pthread_mutex_unlock(&data->mutexEncoderRight);

if(counterLeft-counterRight<-3){
pwmWrite(Pin_D,280);
pwmWrite(Pin_I,330);
}
if(counterLeft-counterRight>3){
pwmWrite(Pin_D,270);
pwmWrite(Pin_I,320);
}
if(counterLeft==counterRight){
pwmWrite(Pin_I,330);
pwmWrite(Pin_D,270);
}
} while (counter >= counterLeft && counter >= counterRight && distLeft >=
security && distRight >= security);

if(distLeft <= security || distRight <= security){
printf("Obstaculo cercano detectado(Rht: %f, Lft: %f. Retirelo cuanto
antes\n",distRight,distLeft);
pwmWrite(Pin_I,300);
pwmWrite(Pin_D,300);
delay(1000);
printf("El vehiculo se acercara al obstáculo a menos de 15 cm.\n");
pwmWrite(Pin_I,320);
pwmWrite(Pin_D,280);
do{
pthread_mutex_lock(&data->mutexLeft);
distLeft = data->distLeft;
pthread_mutex_unlock(&data->mutexLeft);

pthread_mutex_lock(&data->mutexRight);
distRight = data->distRight;
pthread_mutex_unlock(&data->mutexRight);

pthread_mutex_lock(&data->mutexEncoderLeft);
counterLeft = data->counterLeft;
pthread_mutex_unlock(&data->mutexEncoderLeft);

pthread_mutex_lock(&data->mutexEncoderRight);
counterRight = data->counterRight;
pthread_mutex_unlock(&data->mutexEncoderRight);

}while(counter >= counterLeft && counter >= counterRight && distLeft>=15 && distRight>=15);

if(distLeft <= 15 || distRight <= 15){
printf("Objeto demasiado cerca. Procede a ejecutarse la rutina de esqui. (Rht: %f,
Lft: %f.\n", distRight,distLeft);

```

```

pwmWrite(Pin_I,300);
pwmWrite(Pin_D,300);
delay(1000);
modifyCoordinates(actualPosition->x+((float)(counterLeft)+(float)(counterRight))*M_PI*
    diametro/40*sin(actualPosition->theta*M_PI/180),actualPosition->y+((float)(counterLeft)
    +(float)(counterRight))*M_PI*diametro/40*cos(actualPosition->theta*M_PI/180),
    actualPosition->theta, actualPosition);
motorDodgeRoutine(actualPosition, args_ptr);
pwmWrite(Pin_I,300);
pwmWrite(Pin_D,300);
}
}
else{
if(counterLeft<counterRight){
pwmWrite(Pin_D,300);
pwmWrite(Pin_I,320);

do{
pthread_mutex_lock(&data->mutexEncoderLeft);
counterLeft = data->counterLeft;
pthread_mutex_unlock(&data->mutexEncoderLeft);

pthread_mutex_lock(&data->mutexEncoderRight);
counterRight = data->counterRight;
pthread_mutex_unlock(&data->mutexEncoderRight);
//printf("Distancia izquierda: %f, Dist derecha: %f.\n",distLeft,distRight);
//delay(100);
}while(counterLeft<counterRight);}
if(counterLeft>counterRight){
pwmWrite(Pin_D,280);
pwmWrite(Pin_I,300);

do{
pthread_mutex_lock(&data->mutexEncoderLeft);
counterLeft = data->counterLeft;
pthread_mutex_unlock(&data->mutexEncoderLeft);

pthread_mutex_lock(&data->mutexEncoderRight);
counterRight = data->counterRight;
pthread_mutex_unlock(&data->mutexEncoderRight);
//printf("Distancia izquierda: %f, Dist derecha: %f.\n",distLeft,distRight);
//delay(100);
}while(counterLeft>counterRight);}

pwmWrite(Pin_I,300);
pwmWrite(Pin_D,300);

modifyCoordinates(actualPosition->x+((float)(counterLeft)+(float)(counterRight))*M_PI*
    diametro/40*sin(actualPosition->theta*M_PI/180),actualPosition->y+((float)(counterLeft)
    +(float)(counterRight))*M_PI*diametro/40*cos(actualPosition->theta*M_PI/180),
    actualPosition->theta, actualPosition);
}
pthread_mutex_lock(&data->mutexEncoderLeft);

```

```

data->counterLeft=0;
pthread_mutex_unlock(&data->mutexEncoderLeft);

pthread_mutex_lock(&data->mutexEncoderRight);
data->counterRight=0;
pthread_mutex_unlock(&data->mutexEncoderRight);
}

void motorStop(){
printf("Destino Alcanzado\n");
pwmWrite(Pin_I,0);
pwmWrite(Pin_D,0);
//delay(6000);
}

void motorRotateAngle(float angle, void *args_ptr, Position *actualPosition){

printf("Girando el vehículo. El vehiculo se detendrá cuando haya completado un giro
de %fº.\n",angle);
ThreadArgs *data = (ThreadArgs*)args_ptr;
if(angle!=0){
pthread_mutex_lock(&data->mutexEncoderLeft);
data->counterLeft=0;
pthread_mutex_unlock(&data->mutexEncoderLeft);

pthread_mutex_lock(&data->mutexEncoderRight);
data->counterRight=0;
pthread_mutex_unlock(&data->mutexEncoderRight);

int counter = (int)(abs(angle)*eje/360)*cuentas/diametro;
int counterLeft, counterRight;
if(angle<0){
pwmWrite(Pin_I,270);
pwmWrite(Pin_D,270);
}else{
pwmWrite(Pin_I,330);
pwmWrite(Pin_D,330);
}

do
{
pthread_mutex_lock(&data->mutexEncoderLeft);
counterLeft = data->counterLeft;
pthread_mutex_unlock(&data->mutexEncoderLeft);

pthread_mutex_lock(&data->mutexEncoderRight);
counterRight = data->counterRight;
pthread_mutex_unlock(&data->mutexEncoderRight);
} while (counter >= counterLeft && counter>= counterRight);

pwmWrite(Pin_I,300);
pwmWrite(Pin_D,300);

```

```

modifyCoordinates(actualPosition->x, actualPosition->y, actualPosition->theta+angle
/abs(angle)*((float)(counterLeft)/2+(float)(counterRight)/2)*360*diametro/cuentas/eje,
actualPosition);

pthread_mutex_lock(&data->mutexEncoderLeft);
data->counterLeft=0;
pthread_mutex_unlock(&data->mutexEncoderLeft);

pthread_mutex_lock(&data->mutexEncoderRight);
data->counterRight=0;
pthread_mutex_unlock(&data->mutexEncoderRight);
}
}

void *encoderLeft(void *args_ptr){

//SharedData *sharedData = (SharedData *)sharedData;
ThreadArgs *sharedData = (ThreadArgs*)args_ptr;

float t_muestreo=30; //Pasamos el tiempo de muestreo a microsegundos para compararlo con los de
int measure[2]={0,0};
float t1,t2;
//int counterLeft;

while(1) {
t1 = millis();
measure[0]=measure[1];
if(analogRead(pbCntLft)<=40) measure[1] = 0;
if(analogRead(pbCntLft)>=50) measure[1] = 1;
if(abs(measure[1]-measure[0])==1){

pthread_mutex_lock(&sharedData->mutexEncoderLeft);
sharedData->counterLeft++;
//counterLeft=sharedData->counterLeft;
pthread_mutex_unlock(&sharedData->mutexEncoderLeft);
//printf("CounterLeft: %d\n",counterLeft);
}
//printf("MeasureLeft:\t%lf\n",measure[1]);
t2 = millis();
if(t_muestreo>(t2-t1))
delay(t_muestreo-(t2-t1)); //EL tiempo entre medida y medida menos lo que tardamos en leer el pi
if(sharedData->salir) pthread_exit(NULL);
}

return (void *) NULL;
}

void *encoderRigth(void *args_ptr){

//SharedData *sharedData = (SharedData *)sharedData;
ThreadArgs *sharedData = (ThreadArgs*)args_ptr;

```

```

float t_muestreo=30; //Pasamos el tiempo de muestreo a microsegundos para compararlo con los de
int measure[2]={0,0};
float t1,t2;
//int counterRight;
while(1) {
t1 = millis();
measure[0]=measure[1];
if(analogRead(pbCntRth)<=250) measure[1] = 0;
if(analogRead(pbCntRth)>=400) measure[1] = 1;
if(abs(measure[1]-measure[0])==1){

pthread_mutex_lock(&sharedData->mutexEncoderRight);
sharedData->counterRight++;
//counterRight=sharedData->counterRight;
pthread_mutex_unlock(&sharedData->mutexEncoderRight);
//printf("CounterRigth: %d\n",counterRight);
}
//printf("MeasureRigth:\t%lf\n",measure[1]);
t2 = millis();
if(t_muestreo>(t2-t1))
delay(t_muestreo-(t2-t1)); //EL tiempo entre medida y medida menos lo que tardamos en leer el pi
if(sharedData->salir) pthread_exit(NULL);
}

return (void *) NULL;
}

Position *definePosition(){
Position *r = malloc(sizeof(Position));
if(r==NULL){
exit(EXIT_FAILURE);
}
r->x = 0.0;
r->y = 0.0;
r->theta = 0.0;
return r;
}

void modifyCoordinates(float x, float y, float theta, Position *position){
position->x = x;
position->y = y;
position->theta = theta;
printf("Posición actual del robot: x=%f, y=%f, theta=%f\n", position->x, position->y,
position->theta);
}

void motorDodgeRoutine(Position *actualPosition, void *args_ptr){
motorRotateAngle(-90, args_ptr, actualPosition);
motorMoveRelative(0, 400, args_ptr, actualPosition);
motorRotateAngle(90, args_ptr, actualPosition);
motorMoveRelative(0, 500, args_ptr, actualPosition);
}

```



```

void generateInstructions(Position *actualPosition, Position *nextPosition, void *args_ptr,
int security, int *ptr){
printf("Angle1: %f\nDist: %f\nAngle2: %f\n",atan2(nextPosition->x-actualPosition->x,
nextPosition->y-actualPosition->y)*180/M_PI-actualPosition->theta,sqrt((nextPosition->x-
actualPosition->x)*(nextPosition->x-actualPosition->x)+(nextPosition->y-actualPosition->y)*
(nextPosition->y-actualPosition->y)),nextPosition->theta-(atan2(nextPosition->x-
actualPosition->x, nextPosition->y-actualPosition->y)*180/M_PI-actualPosition->theta));
motorRotateAngle(atan2(nextPosition->x-actualPosition->x, nextPosition->y-actualPosition-
>y)*180/M_PI-actualPosition->theta,args_ptr, actualPosition);
delay(100);
motorMoveRelative(security, sqrt((nextPosition->x-actualPosition->x)*(nextPosition->x-
actualPosition->x)+(nextPosition->y-actualPosition->y)*(nextPosition->y-actualPosition->y)),
args_ptr, actualPosition);
if(sqrt((actualPosition->x-nextPosition->x)*(actualPosition->x-nextPosition->x) +
(actualPosition->y-nextPosition->y)*(actualPosition->y-nextPosition->y))>100){
printf("Destino no alcanzado\n");
generateInstructions(actualPosition, nextPosition, args_ptr, security, ptr);}

if(ptr!=NULL){
motorRotateAngle(nextPosition->theta-actualPosition->theta, args_ptr, actualPosition);}
}

void *movement(void *args_ptr){
calDistance(args_ptr);

pwmWrite(Pin_I,300);
pwmWrite(Pin_D,300);

ThreadArgs *data = (ThreadArgs*)args_ptr;
pthread_barrier_wait(&data->barrera);
/**Experimento 0 **/
Position *actualPosition = definePosition();
modifyCoordinates(0,0,0, actualPosition);
motorRotateAngle(360, args_ptr,actualPosition);
motorMoveRelative(0,1290, args_ptr,actualPosition);
free(actualPosition);**/

/**Experimento 1 **/ /*
char commands[] = "D800, R-45, D400";
char *token;
char *delimiter = ", ";
Position *actualPosition = definePosition();
modifyCoordinates(0,0,0, actualPosition);

// Reiniciar strtok para volver a procesar la cadena
token = strtok(commands, delimiter);
char letra;
int valor;
// Procesar la cadena y almacenar los datos en el array Datos
while (token != NULL) {
sscanf(token, "%c%d", &letra, &valor);
switch(letra) {
case 'D':

```

```

        motorMoveRelative(0,valor,args_ptr, actualPosition);
        break;
    case 'R':
        motorRotateAngle(valor, args_ptr, actualPosition);
        break;
    default:
        motorStop();
        printf("Error al leer las órdenes de movimiento. Formato no válido.\n");
        break;
}

    token = strtok(NULL, delimiter);
}

    free(actualPosition);*/

/**Experimento 2 **/ /*
Position *actualPosition = definePosition();
Position *nextPosition = definePosition();
modifyCoordinates(0,0,0, actualPosition);
modifyCoordinates(800,800,-90, nextPosition);
int ptr_value = -1;
generateInstructions(actualPosition, nextPosition, args_ptr, 0, &ptr_value);

free(actualPosition);
free(nextPosition);*/

/**Experimento 3 **/ /*
float camino[] = {-400,400,0,1600,0,0};
int i=0;
Position *actualPosition = definePosition();
Position *nextPosition = definePosition();
modifyCoordinates(0,0,0, actualPosition);
do{
    modifyCoordinates(camino[i],camino[i+1],0, nextPosition);
    i+=2;
} while(i<sizeof(camino)/sizeof(float));
generateInstructions(actualPosition, nextPosition, args_ptr, 15, NULL);
} while(i<sizeof(camino)/sizeof(float));
    free(actualPosition);
    free(nextPosition); */

/**Experimento 4 **/
Position *actualPosition = definePosition();
Position *nextPosition = definePosition();
modifyCoordinates(0,0,0, actualPosition);
modifyCoordinates(0,0,0, nextPosition);
//motorDodgeRoutine(actualPosition, args_ptr);
generateInstructions(actualPosition, nextPosition, args_ptr, 20, NULL);

free(actualPosition);
free(nextPosition);

motorStop();
data->salir = 1;
pthread_exit(NULL);

```

```

return (void *) NULL;
}

int main(int argc, char *argv[]) {

ThreadArgs *sharedData = initSharedData();
//SharedData *sharedData = initSharedData();
int rc;

    wiringPiSetup();

    pinMode(Pin_D,PWM_OUTPUT);
pinMode(Pin_I,PWM_OUTPUT);
pwmSetMode(PWM_MODE_MS);
pwmSetRange(4000);
pwmSetClock(96);


    wiringPiSPISetup(spi_channel,velocidad); //Configuramos canal de comunicación y velocidad
mcp3004Setup (100, spi_channel);
//mcp3004Setup (pbDstRth, spi_channel);

    pthread_t thread1,thread2,thread3,thread4,thread5;

    if ((rc=pthread_create(&thread1,NULL,&distanceMeasureRigth,sharedData))) {
printf("Fallo de pthread:\n");
    }
    if ((rc=pthread_create(&thread2,NULL,&distanceMeasureLeft,sharedData))) {
printf("Fallo de pthread:\n");
    }
    if ((rc=pthread_create(&thread3,NULL,&encoderLeft,sharedData))) {
printf("Fallo de pthread:\n");
    }
    if ((rc=pthread_create(&thread4,NULL,&encoderRigth,sharedData))) {
printf("Fallo de pthread:\n");
    }
    if ((rc=pthread_create(&thread5,NULL,&movement,sharedData))) {
printf("Fallo de pthread:\n");
    }

    pthread_join(thread1,NULL);
    pthread_join(thread2,NULL);
    pthread_join(thread3,NULL);
    pthread_join(thread4,NULL);
    pthread_join(thread5,NULL);
    printf("From main proces IC: %d\n",(int)getpid());

destroySharedData(sharedData);

    return 0;
}

```